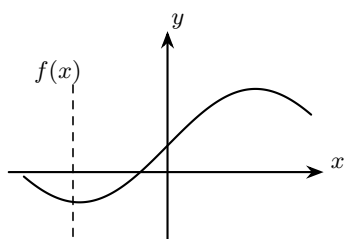


数字电路

(学习 笔记)



魏舟

2026 秋季学期

在这里写入你的格言

目录

第一章	数制与 BCD 码	2
1.1	常用数制	2
1.1.1	常用数制一览	2
1.1.2	二进制与十进制之间的转换	3
1.1.3	二进制算术运算	3
1.1.4	带符号数的表示：原码、反码和补码	3
1.2	几种简单的编码	5
1.2.1	二-十进制码 (BCD 码)	5
1.2.2	格雷码 (Gray 码)	6
1.2.3	计算机字符集	6
第二章	布尔代数	7
2.1	基本逻辑运算	7
2.1.1	与逻辑运算	7
2.1.2	或逻辑运算	8
2.1.3	非逻辑运算	8
2.2	复合逻辑运算	9
2.2.1	与非逻辑	9
2.2.2	或非逻辑	9
2.2.3	与或非逻辑	10
2.2.4	异或逻辑	10
2.2.5	同或逻辑	11
2.3	逻辑电平及正、负逻辑	11
2.4	基本定律和规则	12
2.4.1	逻辑函数的相等	12
2.4.2	基本定律	12
2.4.3	逻辑代数的三条规则	12

前言

本笔记按课程内容整理，目前包括第一章「数制与 BCD 码」、第二章「布尔代数」，其余章节将陆续补入。

整理原则：概念与公式沿用课件原文表述，仅作版式重排；课件中被提取工具打散的公式已按标准写法还原。

第一章 数制与 BCD 码

数制（进位计数制）：用进位的方法来计数，包括**计数符号（数码）**和**进位规则**两个方面。常用的有十进制、二进制、八进制、十六进制等。

1.1 常用数制

1.1.1 常用数制一览

表 1.1: 常用数制一览

数制	计数符号	进位规则	按权展开式
十进制	0 ~ 9	逢十进一	$(N)_{10} = \sum_{i=-m}^{n-1} a_i \times 10^i$
二进制	0, 1	逢二进一	$(N)_2 = \sum_{i=-m}^{n-1} a_i \times 2^i$
八进制	0 ~ 7	逢八进一	$(N)_8 = \sum_{i=-m}^{n-1} a_i \times 8^i$
十六进制	0 ~ 9、A ~ F	逢十六进一	$(N)_{16} = \sum_{i=-m}^{n-1} a_i \times 16^i$

其中 a_i 为系数、 R^i 为权（ R 为基数）。

例：

$$1987.45 = 1 \times 10^3 + 9 \times 10^2 + 8 \times 10^1 + 7 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} \quad (1.1)$$

$$(6D.4B)_{16} = 6 \times 16^1 + 13 \times 16^0 + 4 \times 16^{-1} + 11 \times 16^{-2} \quad (D = 13, B = 11) \quad (1.2)$$

$$(63.45)_8 = 6 \times 8^1 + 3 \times 8^0 + 4 \times 8^{-1} + 5 \times 8^{-2} \quad (1.3)$$

数字电路中采用二进制的原因：

1. 数字装置简单可靠；
2. 二进制数运算规则简单；
3. 数字电路既可以进行算术运算，也可以进行逻辑运算。

1.1.2 二进制与十进制之间的转换

(1) 二进制数转换为十进制数（按权展开法）

$$\begin{aligned}(1011.101)_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= 8 + 2 + 1 + 0.5 + 0.125 = (11.625)_{10}\end{aligned}\quad (1.4)$$

(2) 十进制数转换为二进制数（提取 2 的幂法）

$$\begin{aligned}(45.5)_{10} &= 32 + 8 + 4 + 1 + 0.5 \\ &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} \\ &= (101101.1)_2\end{aligned}\quad (1.5)$$

数制转换还可以采用**基数连除、连乘**（除基数取余、乘基数取整）等方法。

1.1.3 二进制算术运算

算术运算规则与十进制相同，区别是**逢二进一**。其特点是：加、减、乘、除全部可以用**移位**和**相加**两种操作实现，电路结构简化，所以数字电路中普遍采用二进制算术运算。

加法：逐位相加并处理进位。

$$S_0 = A_0 + B_0 \text{ (进位 } C_1), \quad S_1 = A_1 + B_1 + C_1 \text{ (进位 } C_2), \quad \dots \quad (1.6)$$

设 $A = A_3A_2A_1A_0$ 、 $B = B_3B_2B_1B_0$ ，两数之和为 $S = S_3S_2S_1S_0$ ，则

$$A + B = C_4S_3S_2S_1S_0 \quad (1.7)$$

减法：将减数 B 变成补码后与被减数 A 相加，若有进位则舍去，其和即为两数之差。例：

$$\begin{aligned}Y &= (1000)_2 - (0100)_2 = (1000)_2 + [(1011)_2 + 1] \\ &= (1000)_2 + (1100)_2 = (10100)_2 \xrightarrow{\text{舍去进位}} (0100)_2\end{aligned}\quad (1.8)$$

乘法：乘以 2 相当于**左移一位**。方法有：（1）按十进制的乘法过程；（2）采用移位相加的方法。

除法：除以 2^k 相当于**右移 k 位**。例（ $k = 2$ ，即除以 4）：

$$(00001011)_2 \div (100)_2 = (00000010)_2 \text{ (商)}, \quad \text{余数} = (11)_2 \quad (1.9)$$

1.1.4 带符号数的表示：原码、反码和补码

计算机中的带符号数表示为**符号位 + 真值**（合称**机器数**），符号位用“0”表示正、“1”表示负。

定义 1.1 (原码) 真值 X 的原码记为 $[X]_{\text{原}}$ ：最高位为符号位，其余为真值部分；不论数的正负，数值部分均保持原真值不变。

优点：正负号与原真值之间的对应关系简单，容易理解；

缺点：用原码进行加减运算比较困难，**0 的表示不唯一**—— $+0 = 00000000$ ， $-0 = 10000000$ 。

定义 1.2 (反码) 真值 X 的反码记为 $[X]_{\text{反}}$ ：若 $X > 0$ ，则 $[X]_{\text{反}} = [X]_{\text{原}}$ ；若 $X < 0$ ，则符号位不变、数值部分按位取反。**0 的反码也不唯一**： $[+0]_{\text{反}} = 00000000$ ， $[-0]_{\text{反}} = 11111111$ 。

定义 1.3 (补码) 真值 X 的补码记为 $[X]_{\text{补}}$ ：若 $X > 0$ ，则 $[X]_{\text{补}} = [X]_{\text{反}} = [X]_{\text{原}}$ ；若 $X < 0$ ，则

$$[X]_{\text{补}} = [X]_{\text{反}} + 1 \quad (1.10)$$

补码的符号位与 $[X]_{\text{原}}$ 相同，**0 的表示唯一**，且可将减法运算转换为加法运算。

例 ($X = -52 = -0110100$):

$$[X]_{\text{原}} = 10110100, \quad [X]_{\text{反}} = 11001011, \quad [X]_{\text{补}} = [X]_{\text{反}} + 1 = 11001100 \quad (1.11)$$

0 的编码：8 位原码中 $+0 = 00000000$ 、 $-0 = 10000000$ ，不唯一；补码表示唯一：

$$[-0]_{\text{补}} = [-0]_{\text{反}} + 1 = 11111111 + 1 = 100000000 \xrightarrow{\text{8 位字长舍去进位}} 00000000 \quad (1.12)$$

特殊数 10000000:

- 原码中定义为 -0 ;
- 反码中定义为 -127 ;
- 补码中定义为 -128 ;
- 作为无符号数时 $(10000000)_2 = 128$ 。

表示范围 (8 位):

表 1.2: 8 位数的表示范围

编码	表示范围
原码 / 反码	$-127 \sim +127$
补码	$-128 \sim +127$
无符号数	$0 \leq X \leq 2^n - 1$

补码运算：通过引进补码，可将减法运算转换为加法运算，即

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}, \quad [X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} \quad (1.13)$$

例 1.1. 补码加法 $X = -0110100$ ， $Y = +1110100$ ，求 $X + Y$ 。

解.

$$[X]_{\text{补}} = 11001100, \quad [Y]_{\text{补}} = 01110100 \quad (1.14)$$

所以

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 11001100 + 01110100 = 01000000 \quad (1.15)$$

符号位讨论——用二进制补码求 13 ± 10 、 -13 ± 10 (6 位字长):

$$\begin{aligned} 13 + 10 &= 001101 + 001010 = 010111 = 23, \\ 13 - 10 &= 001101 + 110110 = 000011 = 3, \\ -13 + 10 &= 110011 + 001010 = 111101 = -3, \\ -13 - 10 &= 110011 + 110110 = 101001 = -23 \end{aligned} \quad (1.16)$$

无符号数的溢出: 无符号数的范围为 $0 \leq X \leq 2^n - 1$, 若运算结果超出这个范围, 则产生溢出。溢出判断准则: 运算时, 当**最高位向更高位有进位 (或借位)**时则产生溢出。

1.2 几种简单的编码

1.2.1 二-十进制码 (BCD 码)

定义 1.4 (二-十进制码 (BCD 码)) 用四位二进制代码来表示一位十进制数码, 这样的代码称为二-十进制码 (BCD 码, Binary Coded Decimal codes)。四位二进制有 16 种不同组合, 可以在这 16 种代码中任选 10 种表示十进制数的 10 个不同符号, 选择方法不同, 就得到不同的编码形式。常见的有 8421 码、5421 码、2421 码、余 3 码等。

表 1.3: 常用 BCD 码

十进制数	8421 码	5421 码	2421 码	余 3 码
0	0000	0000	0000	0011
1	0001	0001	0001	0100
2	0010	0010	0010	0101
3	0011	0011	0011	0110
4	0100	0100	0100	0111
5	0101	1000	1011	1000
6	0110	1001	1100	1001
7	0111	1010	1101	1010
8	1000	1011	1110	1011
9	1001	1100	1111	1100

定义 1.5 (有权 BCD 码) 每位数码都有确定的位权的码, 例如 8421 码、5421 码、2421 码。

例: 5421 码 1011 代表 $5 + 0 + 2 + 1 = 8$; 2421 码 1100 代表 $2 + 4 + 0 + 0 = 6$ 。

5421 码与 2421 码的编码不唯一, 例如 2421 码 0110 也可表示 6。

由表 1.3 可见:

1. 8421 码与代表 $0 \sim 9$ 的二进制数一一对应;
2. 5421 码前 5 个码与 8421 码相同, 后 5 个码在前 5 个码的基础上加 1000 构成;
3. 2421 码前 5 个码与 8421 码相同, 后 5 个码以中心对称取反, 这样的码称为**自反代码**。例如 $4 \rightarrow 0100$ 、 $5 \rightarrow 1011$; $0 \rightarrow 0000$ 、 $9 \rightarrow 1111$ 。

定义 1.6 (无权 BCD 码 (余 3 码)) 每位数码无确定的位权, 例如余 3 码。余 3 码的编码规律为: 在 8421 码上加 0011。例如 6 的余 3 码为 $0110 + 0011 = 1001$ 。

1.2.2 格雷码 (Gray 码)

格雷码为无权码, 特点为: **相邻两个代码之间仅有一位不同**, 其余各位均相同, 具有这种特点的代码称为**循环码**, 格雷码是循环码。

格雷码与四位二进制码之间的关系: 设四位二进制码为 $B_3B_2B_1B_0$, 格雷码为 $R_3R_2R_1R_0$, 则

$$R_3 = B_3, \quad R_2 = B_3 \oplus B_2, \quad R_1 = B_2 \oplus B_1, \quad R_0 = B_1 \oplus B_0 \quad (1.17)$$

其中 $\oplus$ 为异或运算符, 运算规则: 两运算数相同结果为 “0”, 不同结果为 “1”。

1.2.3 计算机字符集

各种文字和符号的总称, 包括各国家文字、标点符号、图形符号、数字等。常见字符集: ASCII、GB2312、BIG5、GB18030、Unicode。

第二章 布尔代数

数字电路的基础为逻辑代数，由英国数学家 George Boole 在 1847 年提出，逻辑代数也称布尔代数。

2.1 基本逻辑运算

在逻辑代数中，变量常用字母 $A, B, C, \dots, Y, Z, a, b, c, \dots, x, y, z$ 等表示，变量的取值只能是“0”或“1”。

逻辑代数中只有三种基本逻辑运算，即“与”、“或”、“非”。

2.1.1 与逻辑运算

定义 2.1（与逻辑运算） 只有决定一事件的全部条件都具备时，这件事才成立；如果一个或一个以上条件不具备，则这件事就不成立。这样的因果关系称为“与”逻辑关系，其函数式为 $F = A \cdot B = AB$ 。

与门逻辑功能概括：有“0”出“0”，全“1”出“1”。

与门逻辑符号如下：

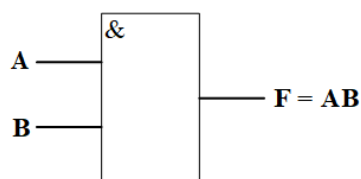


图 2.1: 与门逻辑符号

与逻辑真值表如下：

表 2.1: 与逻辑真值表

A	B	$F = AB$
0	0	0
0	1	0
1	0	0
1	1	1

2.1.2 或逻辑运算

定义 2.2 (或逻辑运算) 在决定一事件的各种条件中，只要有一个或一个以上条件具备时，这件事就成立；只有所有的条件都不具备时，这件事才不成立。这样的因果关系称为“或”逻辑关系，其函数式为 $F = A + B$ 。

或门逻辑功能概括：有“1”出“1”，全“0”出“0”。

或门逻辑符号如下：

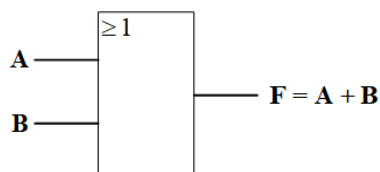


图 2.2: 或门逻辑符号

或逻辑真值表如下：

表 2.2: 或逻辑真值表

A	B	$F = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

注：与门和或门均可以有多个输入端。

2.1.3 非逻辑运算

定义 2.3 (非逻辑运算) 假定事件 F 成立与否同条件 A 的具备与否有关：若 A 具备，则 F 不成立；若 A 不具备，则 F 成立。 F 和 A 之间的这种因果关系称为“非”逻辑关系，其函数式为 $F = \bar{A}$ 。

非门逻辑符号如下：

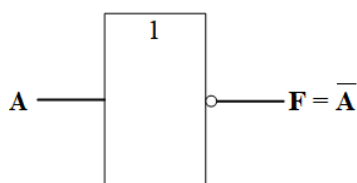


图 2.3: 非门逻辑符号

非逻辑真值表如下：

表 2.3: 非逻辑真值表

A	$F = \overline{A}$
0	1
1	0

2.2 复合逻辑运算

2.2.1 与非逻辑

与非逻辑由与逻辑和非逻辑组合而成，函数式为 $F = \overline{A \cdot B} = \overline{AB}$ 。

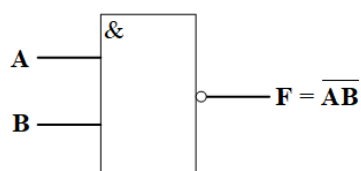


图 2.4: 与非门逻辑符号

与非逻辑真值表如下：

表 2.4: 与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

2.2.2 或非逻辑

或非逻辑由或逻辑和非逻辑组合而成，函数式为 $F = \overline{A + B}$ 。

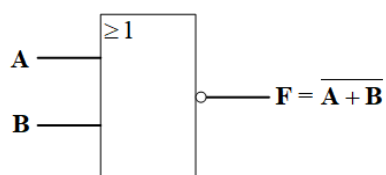


图 2.5: 或非门逻辑符号

或非逻辑真值表如下：

表 2.5: 或非逻辑真值表

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

2.2.3 与或非逻辑

与或非逻辑由与、或、非三种逻辑组合而成，函数式为 $F = \overline{AB + CD}$ 。

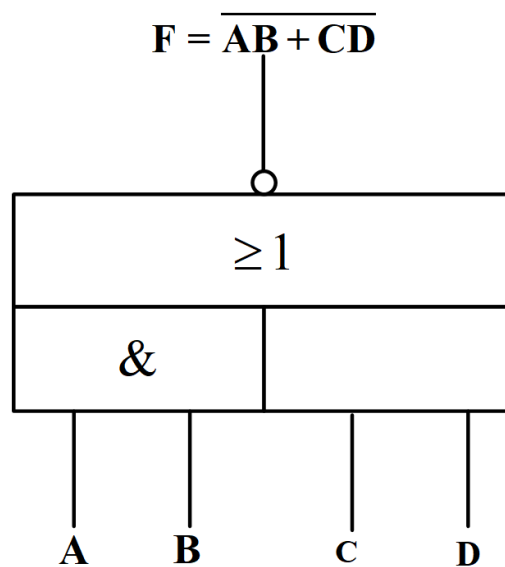


图 2.6: 与或非门逻辑符号

2.2.4 异或逻辑

异或逻辑的函数式为 $F = \overline{A}B + A\overline{B} = A \oplus B$ ，口诀：相同得“0”，相异得“1”。

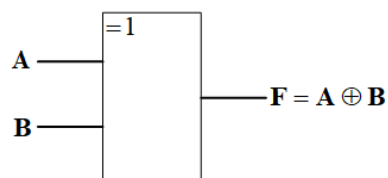


图 2.7: 异或门逻辑符号

异或逻辑真值表如下：

表 2.6: 异或逻辑真值表

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2.2.5 同或逻辑

同或逻辑的函数式为 $F = \overline{A}\overline{B} + AB = A \odot B$ ，口诀：相同得“1”，相异得“0”。

同或和异或互为反函数，即

$$A \oplus B = \overline{A \odot B} \quad (2.1)$$

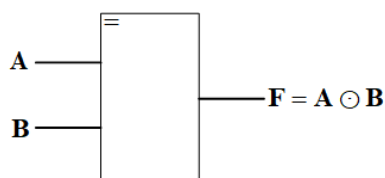


图 2.8: 同或门逻辑符号

同或逻辑真值表如下：

表 2.7: 同或逻辑真值表

A	B	$F = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

2.3 逻辑电平及正、负逻辑

门电路的输入、输出为二值信号，用“0”和“1”表示，这里的“0”、“1”一般用两个不同电平值来表示。

定义 2.4 (正逻辑与负逻辑)

- **正逻辑**：若用高电平 V_H 表示逻辑“1”，用低电平 V_L 表示逻辑“0”，则称为正逻辑约定，简称正逻辑；
- **负逻辑**：若用高电平 V_H 表示逻辑“0”，用低电平 V_L 表示逻辑“1”，则称为负逻辑约定，简称负逻辑；

辑约定，简称负逻辑。
一般采取正逻辑表示。

注： 对一个特定的逻辑门，采用不同的逻辑表示时，其门的名称也就不同。

2.4 基本定律和规则

2.4.1 逻辑函数的相等

设有两个逻辑函数

$$F_1 = f_1(A_1, A_2, \dots, A_n), \quad F_2 = f_2(A_1, A_2, \dots, A_n) \quad (2.2)$$

如果对于 $A_1, A_2, \dots, A_n$ 的任何一组取值（共 2^n 组）， F_1 和 F_2 均相等，则称 F_1 和 F_2 相等。

因此，如两个函数的真值表相等，则这两个函数一定相等。

2.4.2 基本定律

表 2.8: 逻辑代数的基本定律

定律	与形式	或形式
0-1 律	$A \cdot 0 = 0$	$A + 1 = 1$
自等律	$A \cdot 1 = A$	$A + 0 = A$
重叠律	$A \cdot A = A$	$A + A = A$
互补律	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
交换律	$A \cdot B = B \cdot A$	$A + B = B + A$
结合律	$A(BC) = (AB)C$	$A + (B + C) = (A + B) + C$
分配律	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
反演律	$\overline{A + B} = \bar{A} \cdot \bar{B}$	$\overline{AB} = \bar{A} + \bar{B}$

反演律也称**德·摩根定理**，是一个非常有用的定理。

2.4.3 逻辑代数的三条规则

代入规则

任何一个含有变量 x 的等式，如果将所有出现 x 的位置，都用一个逻辑函数式 F 代替，则等式仍然成立。

反演规则

设 F 为任意逻辑表达式，若将 F 中所有运算符、常量及变量作如下变换：

·	+	0	1	原变量	反变量
↓	↓	↓	↓	↓	↓
+	·	1	0	反变量	原变量

图 2.9: 反函数的变换规则

则所得新的逻辑式即为 F 的反函数，记为 $\bar{F}$ 。

注意：

1. 保持原式运算的优先次序；
2. 原式中的不属于单变量上的非号不变。

对偶规则

设 F 为任意逻辑表达式，若将 F 中所有运算符和常量作如下变换：

·	+	0	1
↓	↓	↓	↓
+	·	1	0

图 2.10: 对偶式的变换规则

则所得新的逻辑表达式即为 F 的对偶式，记为 F' 。